ORVIO | PRODUCT REQUIREMENTS DOCUMENT

Stock Management (Inventory Stock Control)

Version 1.0 • Draft for product and engineering review • Prepared 12 October 2026

Status: Proposed feature PRD. This document defines the stock-management product slice; unresolved product rules are explicitly marked OPEN and must be decided before implementation where they affect balances or permissions.

1. Document purpose

This PRD defines how Orvio organizations manage stock quantities across branches. It covers the stock overview, stock movement ledger, opening balances, receiving stock, adjustments, branch transfers, low-stock visibility, POS-driven deductions, permissions, auditability, and phased delivery. It is designed for Orvio's Nigerian small retailers and wholesalers and its existing authenticated, organization-based SaaS model.

This is a feature PRD, not a replacement for the existing Orvio master PRD, API contract, architecture, security, or database specifications. Existing source-of-truth decisions remain in force unless an explicit product/architecture decision changes them.

2. Executive summary

Orvio Stock Management gives owners and staff a reliable view of what is on hand at each branch and a traceable way to explain every quantity change. Users should not silently overwrite a balance: a stock change should be represented by a movement with type, quantity, branch, actor, time, and reason/reference. The balance and movement must be committed together.

MVP supports opening stock, receiving stock without full purchasing workflows, reason-coded adjustments, transfers between branches, sale deductions, stock history, and low/out-of-stock visibility. Physical stock counts and purchase orders/suppliers are later phases.

3. Problem and opportunity

Target merchants often manage stock in notebooks, spreadsheets, or disconnected sales tools. This creates uncertainty about actual stock, weak visibility across locations, untraceable losses, accidental overselling, and slow reconciliation after physical counts. Orvio can provide a simple perpetual stock record in which every supported business event changes branch stock consistently and leaves an audit trail.

4. Goals and non-goals

Goals

- Show current stock by product and branch, including low and zero stock.
- Record and explain each quantity change in a searchable movement history.
- Support common daily stock tasks with fast, understandable flows.
- Prevent invalid negative balances and prevent cross-organization or unauthorized access.
- Keep inventory consistent with POS sales and sale voids using atomic, auditable operations.
- Make a usable MVP for a shop with one or a few branches without requiring purchasing setup.

Non-goals for initial release

- Purchase orders, supplier records, goods-received-note workflow, and supplier payments.
- Batch/lot/serial tracking, expiry/FEFO, manufacturing, kits/BOM, and landed-cost allocation.
- Automated demand forecasting or automatic replenishment/purchase ordering.
- Advanced stock-count scheduling, approval workflows, or accounting valuation methods.
- Offline stock adjustments/transfers. POS offline sale queuing is governed by the POS requirements and must reconcile safely when synced.
- Custom inventory policies and custom roles beyond the existing RBAC model.

5. Users and permissions

Primary users

- Owner: full organization-level stock visibility and control.
- Manager: manage stock at authorized branches and oversee staff activity.
- Storekeeper: view stock and perform permitted receipts/adjustments/transfers within assigned scope.
- Cashier: view sellable stock and create sales; no manual adjustment unless explicitly granted by the existing role rules.
- Accountant: read-only stock and movement visibility, with export if supported by current product permissions.

The PRD permission matrix grants Owner and Manager stock adjustments, allows Storekeeper adjustments within limits, and denies Cashier and Accountant adjustment by default. It permits Manager/Storekeeper transfers. The backend remains authoritative; UI hiding is not security enforcement. Branch-level restrictions must be applied server-side. [Project PRD]

6. Core concepts and business rules

Concept	Requirement
Organization	Tenant boundary. Every stock/product/branch query is organization-scoped from authenticated context; never trust a client-provided org_id.
Branch	Physical store or warehouse where stock is held. On-hand quantity is per product per branch.
Product	Organization-owned catalog item with unique SKU per organization. Stock may be zero at a branch.
Stock balance	Current quantity on hand for one product at one branch. MVP does not permit negative on-hand stock.
Stock movement	Append-only business record for each committed stock increase/decrease or transfer leg. Includes type, quantity, branch, reason/reference, actor, timestamp.
Reason	Required for manual adjustments; selected from supported reason codes and may include a note. Sales/receipts/transfers use system references where appropriate.
Atomicity	Movement creation and balance update occur in one database transaction; failure leaves neither partially committed.
Audit	Movement history is not edited/deleted as a normal user action. Corrections are made by a new compensating movement.
Idempotency	Any retried sale or stock-write request must not apply the same business movement twice; reuse existing idempotency patterns where defined.

7. Scope and feature requirements

7.1 Stock overview/dashboard

- Display total active products, in-stock products, low-stock products, and out-of-stock products for the selected branch or organization view.
- Show a branch selector/filter; clearly label whether totals are for one branch or combined branches.
- Provide actionable links from low/zero-stock indicators to the filtered product list.
- Show loading skeleton, retryable error, and a useful empty state. Empty state prompts product creation or opening-stock entry.
- Do not imply inventory value is available until cost-price completeness and valuation rules are agreed.

7.2 Product stock list

- List products with name, SKU, branch, on-hand quantity, stock status, and optional low-stock threshold.
- Search by name and SKU; barcode search may be used if supported by the existing product data/API.
- Filter by branch, category, active status, low stock, and out of stock where data supports it.

- Use server-side pagination for large lists; maintain organization scope in API and query/cache keys.
- Allow opening a product stock detail/history view.

7.3 Opening stock

- During inventory onboarding or later product setup, allow an authorized user to enter starting quantity for a chosen branch.
- Opening stock creates an auditable initial movement and corresponding stock balance; it is not silently written as an unexplained balance.
- Quantity must be zero or greater; zero may be recorded or omitted according to the onboarding UX decision.
- The UI explains that this is the starting quantity in Orvio, not a purchase transaction.

7.4 Receive stock (MVP, without purchasing)

- An authorized user can increase stock for an existing product at a branch using a “Receive stock” action.
- Capture positive quantity and a reference/note; supplier and purchase-order fields are optional/deferred until purchasing exists.
- Create an inbound movement and update the balance atomically.
- If receiving is not included in the first vertical slice, the product decision must explicitly defer it; opening stock alone is not an ongoing replenishment workflow.

7.5 Stock adjustment

- Authorized user selects product and branch, enters counted/correction quantity or a signed delta according to the final interaction design, selects a reason, and optionally adds a note.
- MVP reasons include damage, loss, found, and correction/other; align reason codes with the existing database enum and resolve any mismatch before implementation.
- Show current balance and resulting balance before confirmation.
- Reject a decrease that would produce a negative quantity.
- Write actor, branch, product, reason, note, prior/resulting quantity or equivalent movement delta, and timestamp.
- For potentially sensitive/high-value adjustments, approval thresholds are deferred; the initial role model still applies.

7.6 Branch transfer

- Authorized user chooses source branch, destination branch, product(s), and positive transfer quantity.
- Source and destination must be different active branches in the same organization.
- Source must have sufficient available stock under the approved availability rule.
- Commit both the outbound and inbound legs as one logical transfer in one transaction; either both occur or neither occurs.
- Keep a shared transfer reference so both legs are visible from history.
- MVP assumes immediate transfer completion; in-transit stock and receive/complete transfer states are deferred unless product discovery requires them.

7.7 Sale deduction and void

- A successful POS sale reduces stock at the sale branch and creates a sale-referenced outbound movement in the same transaction as sale persistence.
- The system rejects or clearly handles a sale that exceeds available stock; no negative quantity.
- A permitted sale void reverses the relevant stock impact through a compensating movement, retains the original movement, requires a reason, and follows POS authorization rules.
- WhatsApp order stock deduction timing must follow the existing order/payment state machine and reservation decision; do not implement a second independent deduction path.
- Retries and duplicate webhooks/order processing must not double-decrement stock.

7.8 Movement history

- Show date/time, product, branch, movement type, quantity delta, resulting/current balance where available, reason/reference, and actor/system origin.
- Filter by date range, branch, product, movement type, and actor where authorized.
- Movement records are append-only for merchant-facing workflows. Corrections happen as new movements.
- Use cursor pagination and stable ordering; protect all endpoints with authentication, authorization, validation, and org scoping.

7.9 Low-stock visibility

- Allow a per-product threshold, or a clearly defined global default if product-level threshold is not in the MVP.
- Mark low stock when quantity is at/below the configured threshold; mark out of stock when quantity is zero. Exact comparison is OPEN pending confirmation.
- Show in-app dashboard/list indicators. Email/push notifications are deferred unless notification infrastructure is already available.
- Avoid duplicate alert spam; notification event behavior is a later decision.

8. User journeys

Journey A — Owner opens stock dashboard

1. Owner selects organization and Inventory.
2. Dashboard loads branch-aware stock summary and low/out-of-stock items.
3. Owner filters to a branch or searches for a product.
4. Owner opens a product and reviews quantity and movement history.

Journey B — Storekeeper receives stock

5. Storekeeper selects authorized branch and Receive stock.
6. Selects product, enters positive quantity and optional reference/note.
7. Reviews resulting quantity and confirms.
8. System atomically records movement, updates balance, and displays confirmation.

Journey C — Adjust damaged/lost stock

9. Authorized user selects product/branch and Adjust stock.

10. Enters decrease/increase, selects reason, and adds note if useful.
11. System shows current and resulting balance.
12. User confirms; system commits movement and balance together and exposes it in history.

Journey D — Transfer between branches

13. Authorized user chooses source and destination branches.
14. Adds one or more products and quantities.
15. System validates branch scope and available stock, then presents transfer summary.
16. On confirmation, both transfer legs commit atomically and appear in source/destination histories.

Journey E — POS sale

17. Cashier completes a sale at a branch.
18. Backend validates stock and permissions.
19. Sale, stock decrement, and movement commit atomically and idempotently.
20. Updated stock appears in dashboard/history; void follows the approved reversal flow.

9. Functional requirements and priority

ID	Requirement	Priority	Acceptance summary
STK-001	View stock by organization/branch	Must	Authorized user sees correct branch-scoped quantities; no cross-tenant data.
STK-002	Stock dashboard counters and alerts	Must	Counts match underlying active product/stock data and filters.
STK-003	Search/filter/paginate product stock	Must	Search by name/SKU; branch and stock-state filters work server-side.
STK-004	Record opening stock	Must	Creates balance and auditable opening movement; non-negative only.
STK-005	Receive stock	Should for first stock slice	Positive receipt increases balance and creates traceable inbound movement.
STK-006	Manual stock adjustment	Must	Reason required; actor recorded; no negative result; atomic write.
STK-007	Transfer stock between branches	Must	Same-org distinct branches; sufficient stock; atomic paired movements.
STK-008	Sale stock deduction	Must for POS integration	Sale and stock movement are atomic/idempotent; oversell behavior enforced.
STK-009	Void/reversal stock handling	Must with POS void	Compensating movement; original history retained; reason and permission required.
STK-010	Movement history	Must	Filterable, cursor-paginated, auditable, append-only.
STK-011	Low-stock threshold	Must basic visibility	Threshold source and boundary rule are configured consistently.
STK-012	Physical stock count/reconciliation	Later	Count, variance review, and adjustment generation.
STK-013	Purchase order/supplier/GRN	Later	Full purchasing workflow and receipt references.
STK-014	Reorder suggestion/forecasting	Later	Based on agreed sales/demand data and policy.

10. Data and movement model

Use the existing Orvio conceptual entities: products, stock_items (product/branch balance), and stock_movements (ledger), linked to organization, branch, and actor. The database specification already defines org-scoped products and stock, non-negative on-hand/reserved constraints, and movement types/reasons. Review the implemented Prisma schema before adding fields or enums; update database documentation and migrations together if changes are required. [Project database specification]

Movement event	Stock effect	Reference / notes
Opening balance	Increase from zero or prior tracked balance	Opening/setup reference; actor recorded.
Receipt	Increase at destination branch	Supplier/PO optional until purchasing; receipt reference/note.
Adjustment increase/decrease	Signed change at one branch	Required reason and optional note.
Transfer out	Decrease at source	Paired to same transfer identifier.
Transfer in	Increase at destination	Paired to same transfer identifier.
Sale	Decrease at sale branch	Linked to sales order and idempotency key/reference.
Sale void/reversal	Increase back at branch as compensating movement	Linked to original sale and void; do not delete original.

Important schema alignment: the existing database document describes movement types in/out/transfer/adjust, signed quantities, reason enums, and stock balance per branch. It also says movement records are never deleted. Implement transfer pairing and opening/receipt semantics using that model where possible; any incompatible change requires a documented schema/API decision. [Project database specification]

11. API and system behavior

Follow existing REST `/api/v1` conventions, response envelopes, Fastify route → service → repository boundaries, Zod validation, Prisma transactions, and existing auth middleware. Do not have the frontend write directly to the database. The endpoint names below are conceptual and must be reconciled with the existing API contract before build.

Capability	Conceptual operation	Key validation
Dashboard summary	GET stock summary by branch/filter	Auth, org membership, branch scope; computed counts.
Stock list	GET stock items with filters/cursor	Org/branch scope, allowed filters, pagination.
Movement history	GET stock movements with filters/cursor	Org scope, branch authorization, stable order.
Opening/receipt/adjustment	POST stock movement action	Permission, product/branch ownership, quantity, reason, idempotency as appropriate.
Transfer	POST stock transfer	Distinct same-org branches, positive quantities, sufficient availability, atomic paired entries.
POS sale/void integration	Existing POS write path invokes stock domain service	Atomic transaction, permission, availability, idempotency, compensating reversal.

12. Validation and error handling

- Return a clear validation error for invalid quantity, missing reason, duplicate/invalid resource, inactive branch, or product not belonging to the organization.
- Use a conflict/insufficient-stock response when the resulting balance would be negative or source stock is insufficient.
- On concurrent updates, lock or conditionally update the balance so two requests cannot both spend the same available stock. Return a retryable conflict rather than silently accepting an invalid balance.
- On database/API failure, leave both balance and movement unchanged; provide a safe retry path.

- Errors must follow Orvio’s standard error envelope and not disclose internal details.

13. Security, audit, and data integrity

- Require authenticated user and active organization membership for all protected stock routes.
- Enforce role permissions and branch scope server-side for reads and writes.
- Derive organization and actor IDs from authenticated server context; do not trust request body identity fields.
- Include org_id in repositories and all queries; test cross-org and cross-branch IDOR attempts.
- Validate request schemas at API boundary; use parameterized ORM access.
- Commit balance changes, movement records, and related sale/transfer records within transactions.
- Preserve immutable movement history; prohibit ordinary editing/deleting of posted movements.
- Record actor, request correlation ID and relevant audit metadata using existing audit/logging conventions without logging secrets.

14. Non-functional requirements

- Performance: Stock dashboard should meet Orvio’s general dashboard target of under 2 seconds for a typical organization; measure and tune with realistic data.
- Consistency: All balance-affecting workflows use a single stock domain service or equivalent shared business rule; POS and future channels must not implement independent stock math.
- Reliability: Movement and balance updates are transactional; retried operations are idempotent where applicable.
- Usability: Desktop-first, tablet-friendly; clear quantities and branch labels; accessible keyboard focus and form errors.
- Scale: Server-side filtering/pagination; indexes follow observed query patterns and existing database guidance.
- Availability: If stock read fails, show a retryable error. Never show stale/optimistic stock as confirmed after a failed write.

15. Analytics and success measures

Metric	Definition / purpose
Stock setup completion	Organizations that have at least one branch/product with an opening balance; assess onboarding effectiveness.
Stock activity adoption	Organizations with at least one stock movement in a defined period; separates setup from ongoing usage.
First sale activation	Organizations with at least one product and one sale within seven days, consistent with existing PRD activation metric.
Low-stock exposure	Count of product/branch combinations flagged low or zero; use as product usage/visibility indicator, not a business outcome by itself.
Adjustment rate	Manual adjustments relative to total movements; investigate unusually high rates as possible UX or process issue.
Transfer completion/error rate	Track attempted/committed transfers and validation failures.
Integrity incidents	Any negative stock, duplicate movement, unmatched transfer leg, or sale/balance mismatch; target zero.

Do not set numeric conversion targets until baseline/pilot data exists. Product owner should define the measurement window and cohort rules before analytics implementation.

16. Rollout and delivery plan

Milestone	Scope	Exit gate
M0 — Discovery/API alignment	Inspect current code/schema and API contract; confirm movement semantics, availability, reason codes, permissions, receipt and low-stock policies.	Open decisions affecting data integrity resolved; API/data changes documented.
M1 — Stock read foundation	Stock summary/list, branch filter, product stock detail, empty/loading/error states.	Counters and lists reconcile to database; authZ and tenant isolation tests pass.
M2 — Opening stock + adjustments	Opening balance and reason-coded manual adjustments through one stock service.	Atomic movement/balance writes; non-negative rule; integration tests and audit evidence.
M3 — Transfers + receipts	Multi-branch transfers and receive-stock flow.	Paired transfer movements commit atomically; branch scoping and concurrency tests pass.
M4 — POS integration	Sale deduction, idempotency, stock validation, void compensation.	E2E product → stock → sale → void; no double deduction on retry.
M5 — Pilot/hardening	Observe real merchant workflows, review errors and adjustment patterns, tune dashboard and empty states.	No critical stock integrity/security defects; pilot feedback reviewed.

Frontend and backend can work in parallel after the API contract stabilizes. Backend stock domain/API, frontend dashboard/action flows, and QA tests should converge in vertical slices. This follows Orvio's roadmap principle to build incremental end-to-end slices and include security/testing in each phase. [Project roadmap]

17. Test requirements and definition of done

Unit tests

- Movement sign/balance calculations and reason validation.
- Low/out-of-stock threshold calculations.
- Transfer pairing and sale reversal rules.

API/database integration tests

- Valid/invalid opening stock, receipt, adjustment, transfer, sale, and void flows.
- Transaction rollback on injected failure; no balance-only or movement-only writes.
- Concurrency test for two decreases against the same stock balance.
- Idempotency/retry test for sale and repeated request.
- RBAC, branch scope, cross-organization IDOR, and user-with-no-membership denial.
- Pagination/filter correctness and dashboard summary reconciliation.

End-to-end tests

- Onboarding/opening stock → stock dashboard → adjustment → history.
- Transfer from one branch to another and verify both balances/history.
- POS sale reduces stock; void restores via a new movement.
- Empty/error/loading states and denied permission experience.

Definition of done

- Requirement and API contract implemented; schema docs/migration updated if needed.
- Backend authZ, tenant isolation, input validation, and audit metadata in place.
- Unit, integration, and affected E2E tests pass.
- Loading, empty, success, validation, and error states are implemented.
- No negative stock, duplicate movement on retry, or partial transfer under tested scenarios.
- Code review and staging verification completed; feature telemetry and support-visible errors considered.

18. Risks and mitigations

Risk	Impact	Mitigation
Stock balance diverges from movement ledger	Merchant loses trust; sales/stock reports wrong	One write service, transactional writes, reconciliation tests and operational integrity alerts.
Concurrent sales oversell	Negative or incorrect inventory	Atomic conditional update/locking; enforce database check; conflict response.
Duplicate retry or webhook processing	Double deduction/addition	Idempotency keys/references and unique processing guards.
Transfer only commits one leg	Stock disappears or duplicates	One transaction with linked source/destination legs.
Unclear availability/reservation semantics	Sales incorrectly accepted/rejected	Resolve reserved quantity and WhatsApp timing before POS/WA integration.
Too much MVP scope	Delayed launch	Defer purchasing, forecasting, and formal stock counts.
Permission mismatch by role	Unauthorized stock changes	Confirm role matrix, enforce server-side, test each permission and branch scope.

19. Open decisions requiring product owner confirmation

- Opening-stock and adjustment UX: enter a signed quantity delta, or enter counted/resulting quantity and let the system derive the delta? Recommendation: adjustment is based on an explicit increase/decrease delta with current and resulting balance shown; stock count reconciliation later uses counted quantity.
- Low-stock threshold: per product globally, per product per branch, or global default? Existing PRD says per product or global threshold. Recommendation: per product per branch for correctness, but validate MVP complexity.
- Low-stock comparison: alert when quantity is below threshold (<) or at/below threshold (≤)? Existing PRD uses both wordings in different places; choose one.
- Can users sell below zero? Existing PRD says no. Confirm as strict MVP rule, including offline POS synchronization conflict behavior.
- How should reserved_quantity affect sellable stock, especially pending WhatsApp orders? Reservation is described as optional in existing PRD; select no reservation or a defined reservation lifecycle before implementing related flows.
- Transfer semantics: immediate branch-to-branch move or in-transit transfer with separate dispatch/receipt? Recommendation: immediate atomic move for MVP.
- Stock unit precision: integer quantity only or fractional quantities for kg/litre/metre? Existing schema specifies integer quantity; confirm target merchant categories before locking.
- Receipt movement: include a simple Receive stock action in this slice, or defer until purchasing/GRN? Recommendation: include simple receipt because merchants need a replenishment workflow even without supplier management.
- Manual adjustment reasons: finalize allowed enums and whether “other” requires a note; align PRD and database enum.
- Approval threshold: any manager approval for high-value/high-quantity adjustment? Recommendation: defer; retain role limits and full actor/reason history for MVP.
- Cost and stock valuation: do not show inventory valuation until cost-price completeness and valuation method (e.g., average cost/FIFO) are decided.
- Low-stock notification delivery and alert deduplication: dashboard indicator only for MVP or in-app event/notification as well? Recommendation: dashboard/list only initially.
- CSV import scope: product-only, opening stock included, or both? Ensure imported initial balances create movements and report partial failures clearly.

20. Traceability to Orvio project requirements

This PRD derives from Orvio’s existing product requirements for products, branch stock, adjustments, transfers, low-stock alerts, sale-driven stock deduction, roles, tenant isolation, and audit logging. The database specification defines products, stock_items, and stock_movements with org scoping, non-negative stock, and append-only movement history. The architecture and implementation plan require the modular-monolith layering, backend authorization, transactional integrity, tests, and end-to-end inventory flows. [Project PRD] [Project database specification] [Project architecture] [Project implementation plan]

Before implementation, inspect current repository and API contract to determine which of these requirements already exist, avoid duplicate models/endpoints, and record any change to established schema or decisions. This follows the Orvio roadmap and decision ledger. [Project roadmap] [Project decision ledger]